

CST8219 – C++ Programming

Project: Personal Expense Tracker and Budget Management System

I. Objective

Design and implement a **Personal Expense Tracker and Budget Management System** using **C++** and the **Qt GUI library**. This project evaluates your ability to apply:

- Classes, inheritance, and abstract classes
- Constructors, destructors, and memory management
- Templates and generic programming
- Operator overloading and friend functions
- GUI programming using **Qt Widgets**
- Data persistence using **file input/output**

You will develop a **GUI-based application** that allows users to record expenses and income, manage budgets, categorize transactions, and view financial analytics such as total spending, savings, and category-wise summaries.

II. Reference:

- **Qt Documentation:** <https://doc.qt.io/>
- C++ Standard Library: <https://en.cppreference.com/w/>
- **JSON for Modern C++:** <https://github.com/nlohmann/json>
- Algonquin College Academic Integrity Policy: <https://www.algonquincollege.com/policies/aa48/>

III. Project Description

The Expense Tracker application should allow users to:

- Add, edit, and delete income and expense records\
- Categorize transactions (e.g., Food, Rent, Transport, Entertainment)
- Set monthly budgets per category
- Search and filter transactions by date, category, or amount
- View summaries and analytics such as total expenses, income, savings, and budget usage
- Save and load all data from files

System Entities

1. Transaction (Abstract Class)

Attributes:

- transactionID
- date
- amount
- category
- description

Behaviors:

- computeImpact() = 0
- displayTransaction()

2. Expense (Derived from Transaction)

Additional Attributes:

- paymentMethod

Behaviors:

- computeImpact()
- displayTransaction()

3. Income (Derived from Transaction)

Additional Attributes:

- source

Behaviors:

- computeImpact()
- displayTransaction()

4. Category

Attributes:

- categoryID
- name
- monthlyBudget

Behaviors:

- updateBudget()
- displayCategorySummary()

5. TransactionManager

Responsibilities:

- Maintains collections of transactions and categories

Behaviors:

- addTransaction()
- editTransaction()
- deleteTransaction()
- searchTransactions()
- filterByCategory()
- filterByDateRange()

6. AnalyticsEngine (Template Class)

Responsibilities:

- Calculate financial summaries

Behaviors:

- computeTotalExpenses()
- computeTotalIncome()
- computeSavings()
- computeCategorySpending()
- computeBudgetUsage()

7. DataManager

Responsibilities:

- Save and load transactions and categories

Behaviors:

- saveData()
- loadData()

8. GUI (Qt Widgets)

- Tabs or menus for: **Transactions | Categories | Budgets | Analytics**
- Forms for adding/editing income and expenses
- Tables or lists to display transaction history
- Charts or summaries for analytics

IV. Coding Constraints:

Language and Compiler

- C++17 or later
- GCC (MinGW) or MSVC
- Code must compile without errors or warnings

OOP Design

- Inheritance for transaction types
- At least one abstract class with pure virtual function
- Constructor chaining and destructors
- Use smart pointers (unique or shared)
- Private/protected members with getters/setters

Templates and Operator Overloading

- Template class/function for analytics
- Overload at least two operators (e.g., <<, ==)
- At least one friend function

Standard Library

- Use STL containers (vector, map, unordered_map)
- Avoid raw arrays

File I/O / Persistence

- Save/load transactions and categories using JSON or text files
- File paths configured through GUI

GUI Constraints

- Must use Qt Widgets
- Tabs or menus for main features
- Forms for data entry
- Tables/lists for displaying records
- Handle invalid input gracefully

Code Quality

- Consistent naming conventions
- Comments for classes, functions, and logic
- Modular code: each class in .h and .cpp

V. Input Validation and Edge Cases:

Transaction Validation

Students must validate all user inputs before creating or updating a transaction.

Transaction ID Validation

- IDs must be unique.
- IDs cannot be empty.
- IDs must follow the format:

TRX-0001

TRX-0002

TRX-0003

Use regular expressions for validation.

Amount Validation

- Must be numeric.
- Must be greater than zero.
- Maximum value: \$1,000,000.
- Reject negative values.
- Reject invalid numeric formats.
- Examples:

100.50	Valid
-25.00	Invalid
abc	Invalid

Date Validation

- Cannot be empty.
- Must follow: YYYY-MM-DD
- Must represent a valid calendar date.
- Future dates are not allowed.
- Examples:

2026-06-01	Valid
2026-13-01	Invalid
2026-02-31	Invalid

Description Validation

- Cannot be empty.
- Minimum length: 5 characters.
- Maximum length: 100 characters.
- Must contain at least one alphabetic character.
- Leading and trailing spaces should be trimmed automatically.

Category Validation

○ Category Name

- Cannot be empty.
- Maximum length: 30 characters.
- Categories must be unique (case-insensitive).
- Example:

Food
food
FOOD

must be treated as the same category.

Budget Validation

- Budget must be numeric.
- Budget must be positive.
- Budget cannot exceed \$1,000,000.
- Reject invalid numeric input.

Search Validation

- Empty searches return all transactions.
- Search must be case-insensitive.
- Invalid search input must not crash the application.
- Special characters must be handled safely.

File Validation

When loading transaction data: Students must:

- Verify that the file exists.
- Verify that the file can be opened.
- Detect corrupted JSON or text data.
- Detect duplicate transaction IDs.

- Detect duplicate category IDs.
- Handle missing required fields gracefully.
- Display meaningful error messages.
- Example:


```

      {
          "amount": 150.00
      }
      
```
- Missing:
 - ❖ transactionID
 - ❖ date
 - ❖ category
 - ❖ description

must trigger validation errors.

Category Budget Validation

Students must detect:

- Transactions exceeding the category budget.
- Budget utilization over 100%.
- Categories with zero budgets.

Display warnings when limits are exceeded.

Additional Edge Cases

GUI Edge Cases: Students must handle:

- Pressing Save multiple times rapidly.
- Editing while another dialog is open.
- Closing the application with unsaved changes.
- Attempting to edit without selecting a transaction.
- Attempting to delete without selecting a transaction.

Data Edge Cases: Students must test:

- Thousands of transactions.
- Very large transaction amounts.
- Extremely long descriptions.
- Duplicate transactions loaded from files.
- Categories without budgets.

File Edge Cases: Students must handle:

- Missing file.
- Empty file.
- Read-only file.
- Corrupted JSON file.
- Invalid character encoding.

Exception Handling Requirements: Students must use:

```
try
{
    // file operation
}
catch(const std::exception& ex)
{
    QMessageBox::critical(...);
}
```

- Handle:
 - ❖ File opening failures
 - ❖ JSON parsing errors
 - ❖ Invalid data conversion
 - ❖ Memory allocation failures

Enhanced Validation Features

Students must implement at least THREE of the following.

Feature 1 – Real-Time Validation: As users type:

- Invalid fields highlighted in red.
- Validation messages displayed immediately.

Feature 2 – Duplicate Detection: Warn users when:

- Transaction IDs already exist.
- Duplicate categories are created.

Feature 3 – Budget Warning System: Display warnings when:

- Budget Usage > 80% and Budget Usage > 100%

Feature 4 – Auto-Save Validation

- Prevent saving when validation fails.

Feature 5 – Unsaved Changes Detection:

- Prompt:

You have unsaved changes.
Do you want to save before exiting?

Feature 6 – Spending Alert: Display warning when a single transaction exceeds a configurable threshold.

- Example:

Warning:
This expense exceeds \$1000.
Are you sure you want to continue?

VI. Minimum Requirements:

1. OOP Design

- ✓ Abstract base class (Transaction)
- ✓ Inheritance (Expense, Income)
- ✓ Constructor chaining and destructors
- ✓ Smart pointers
- ✓ STL containers

2. Templates & Operators

- Template class/function for analytics
- Overload operators (<<, ==)
- Include at least one friend function

3. GUI (Qt)

- Main window with tabs or menus
- Forms for adding/editing transactions
- Data list/table view
- Analytics display

4. Data Persistence

- Save/load all data to JSON or text files

- Load previous data on startup

5. Input Validation

- Proper validation
- Edge case handling

6. Modern C++ Features

- Use at least one `std::optional` or `std::expected`
- Use range-based for loops
- Use smart pointers for dynamic memory management

VII. Development Steps

1. Design class structure & relationships (UML diagram below)
2. Implement Transaction, Expense, and Income classes
3. Develop TransactionManager logic
4. Implement AnalyticsEngine
5. Integrate Qt GUI
6. Implement file save/load functionality
7. Test with multiple users, categories, and budgets

VIII. Suggested Folder Structure:

```
ExpenseTrackerApp/  
├── include/  
│   ├── Transaction.h  
│   ├── Expense.h  
│   ├── Income.h  
│   ├── Category.h  
│   ├── TransactionManager.h  
│   ├── Analytics.h  
│   └── DataManager.h  
├── src/  
│   ├── main.cpp  
│   ├── Transaction.cpp  
│   ├── Expense.cpp  
│   ├── Income.cpp  
│   ├── TransactionManager.cpp  
│   ├── DataManager.cpp  
│   └── GUI.cpp  
└── data/
```



IX. UML Class Diagram

Transaction (Abstract)

- **transactionID : string**
 - **date : string**
 - **amount : double**
 - **category : string**
 - **description : string**
-
- **computeImpact() = 0**
 - **displayTransaction()**

Expense : Transaction

Income : Transaction

Category

- **categoryID**
 - **name**
 - **monthlyBudget**
-
- **updateBudget()**
 - **displayCategorySummary()**

TransactionManager

transactions : vector<unique_ptr<Transaction>>
categories : vector<unique_ptr<Category>>

- **addTransaction()**
- **editTransaction()**
- **deleteTransaction()**
- **searchTransactions()**

AnalyticsEngine<T> must be implemented as a template class or contain at least one template function operating on generic transaction collections.

-
- **computeTotalExpenses()**
 - **computeTotalIncome()**

- `computeSavings()`
- `computeBudgetUsage()`

X. Sample Qt GUI Mockups

1. Main Window

- Tabs: Transactions | Categories | Budgets | Analytics
- Buttons: Add, Edit, Delete

2. Transaction Form

- Fields: ID, Date, Amount, Category, Description
- Buttons: Save, Cancel

3. Category/Budget Form

- Fields: Category Name, Monthly Budget
- Buttons: Save, Cancel

4. Analytics View

- Total income
- Total expenses
- Savings
- Budget usage per category